AI 即编译器：不经过 Triton 编译器，直接编译 Triton Kernel

arXiv:2609.36800v1 [cs.AI] 2026 年 9 月 29 日

François Costa*（斯坦福大学）

Charly Castes（洛桑联邦理工学院）

Thomas Bourgeat（洛桑联邦理工学院）

Azalia Mirhoseini（斯坦福大学）

*通讯作者：francois.costa@stanford.edu

本文是对 arXiv:2609.36800v1 的非官方中文译本，供阅读。公式、专名、实验数字与参考文献保持与原文一致。图中的曲线与原始 PDF 相同，此处只译图题与表题。

摘要

随着编程模型、工作负载和加速器不断变化，编译器后端的构建和维护代价很高。我们研究大语言模型能否取代传统的优化与 lowering 流水线，并把这一过程称为 **AI lowering**。我们考察从 Triton 到 NVIDIA PTX 的 AI lowering：由 LLM 智能体把 Triton kernel 直接翻译成 PTX。我们构造了一个评估候选 PTX 的环境，以及一套由 LLM 把 Triton kernel 译成 PTX 的智能体框架。在 Ada、Hopper、Blackwell 三种 GPU 上的 12 个常见 kernel，以及近期机器学习论文中的 10 个 kernel 上，AI lowering 达到自动调优后 Triton 性能的 **0.83× 到 3.34×**。最大的收益来自 Triton 的 lowering 流水线不做的变换，例如把打包的二值权重直接解码成 Tensor Core 操作数（BitDelta 上 3.34×），让每个线程在张量内存中独占一整行 softmax（FlashAttention 上 1.37×），以及复用重叠的卷积窗口（最高 2.23×）。这些结果依赖于一套带有完整校验支持的稳健评估框架。我们以现有的 PTX 验证器 Volta 为基础，并大幅扩展它，使其支持现代 GPU 架构，包括 Blackwell 的 tcgen05 Tensor Core 接口。这需要为三种架构特性建模：受管理的张量内存、基于描述符的操作数布局，以及由 commit、wait、内存屏障和 proxy fence 协调的异步执行。我们讨论了把它们形式化所面临的困难，以及目前的局限。结果表明，一种新的可能正在出现：AI 编译器取代手写的中间表示和检查器，从而降低为新的通用芯片和定制芯片建立软件栈所需的时间与工程量。

1 引言

高性能 GPU kernel 需要大量针对硬件的专门知识，因此随着工作负载和架构演进，开发、优化和维护的成本都很高。Triton、TileLang、CuTe DSL、ThunderKittens、HipKittens、ParallelKittens 等领域专用语言，通过暴露面向 tile 或感知硬件的抽象，减轻了 kernel 程序员的负担。但它们大体上只是把复杂性挪进了编译器：要支持新的抽象和硬件特性，仍然必须实现 lowering、调度、合法性检查和优化。传统编译器后端进一步加重了这一负担，因为它们依赖多层中间表示（IR），而这些 IR 的语义、变换和 lowering 路径必须持续扩展和维护。MLIR 一类系统试图减少重复的基础设施，但 LLVM、XLA 等大型生产编译器仍然说明：在不引入正确性或性能回退的前提下，维护庞大的优化与代码生成流水线有多难。

随着加速器架构更加多样、演进更快，这个问题变得更严重。GPU、TPU、Trainium、晶圆级系统、LPU、存内计算处理器和定制 AI ASIC，暴露出越来越不同的执行、内存和通信模型；即便同一加速器家族的相邻代际，也会引入现有 IR 无法自然表达的能力。于是新系统越来越多地引入专门抽象和编译路径，例如 Tawa 用于 warp 特化的异步引用，或 ThunderKittens 针对硬件的 tile 与 warp 抽象。硬件创新的速度，使以 IR 为中心的传统编译器开发越来越贵，也越来越难以与当前最好的性能保持对齐。当现有编译器抽象不能高效暴露新硬件时，性能关键的实现就会绕过 IR，改用目标专用的 intrinsic 或汇编。DeepGEMM 用 Hopper 专用的 PTX 指令，就是如此。

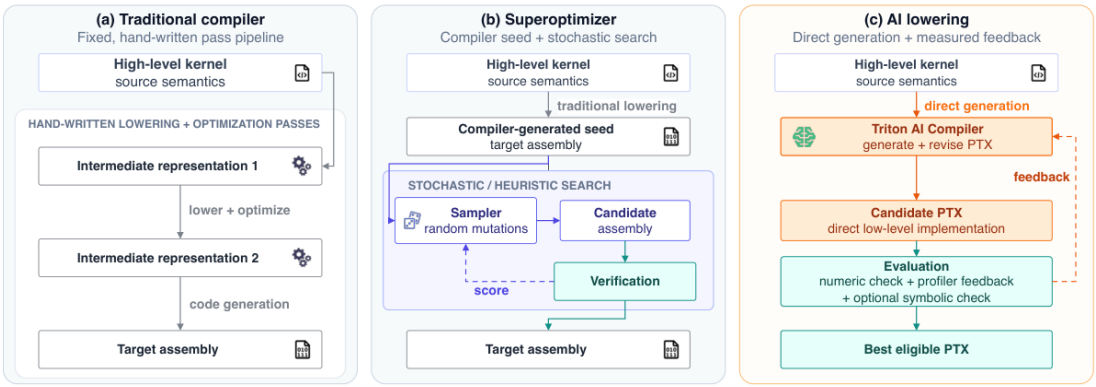


图 1. 传统编译、超优化与 AI lowering。(a) 传统编译器在中间表示上应用手写的 lowering 与优化遍。(b) 超优化器以传统 lower 出的目标代码为种子，然后迭代地变异候选程序并打分。(c) AI lowering 绕过传统的优化 / lowering 后端：LLM 直接生成 PTX，并利用数值与性能反馈进行修改，可选地再做符号化验证。在基于搜索的方法中，接受与否由数值检查或形式化检查把关。

我们表明：对关键的 GPU kernel 而言，绕过传统 lowering 流水线不仅可行，而且有益。我们提出 **TRITON AI COMPILER（TAIC）**，一条直接的 Triton 到 PTX 编译路径，实现 AI lowering：LLM 从 Triton 源码直接合成候选 PTX。这与传统编译器不同。传统编译器把高层代码逐步 lower 到多个人工设计的中间表示。我们观察到，AI lowering 能以非平凡的方式优化 PTX，而这些方式是基线 Triton 编译器接触不到的，例如把 Tensor Core 指令与 warp 级 shuffle 结合起来，在 PTX 层直接重组数据搬运和计算，以及选用 Triton 所评估的那条 lowering 流水线没有暴露的、针对目标的指令。在 Ada、Hopper、Blackwell 上，AI lowering 达到 Triton 所生成、并经过优化的 PTX 的 0.83× 到 2.23×。在 B200 上，它也超过了广泛使用的、已经优化过的 Triton kernel，其中包括 FlashAttention 的 1.37×，以及一个 Mamba-2 原语的 1.10×。

生成本质上是概率性的，正确性却不该如此。现有验证工具只覆盖了现代 GPU 执行的一小部分。PTX 验证的当前最好结果 Volta，会符号化地执行一个 CTA 中的每个线程，并检查竞争、死锁和输出表达式，但它主要支持寄存器、共享内存、warp 原语、批量同步屏障、Hopper 之前的指令，以及与输入无关的控制流。现代 kernel 超出了这一片段，而 AI lowering 把缺口拉得更大：它使用的 PTX 指令变体超过 100 种，这些变体并不出现在 Triton 编译器原来的输出里。若把 AI lowering 限制在验证器支持的子集上，其性能优势就会消失。因此，我们把「快速演化的 GPU ISA」与「已形式化的语义」之间的缺口，视为 AI kernel 生成的新核心难题。本着这一看法，我们为 Volta 建立了支持 Hopper 和 Blackwell 的扩展，其中包括集成 Blackwell 的 tcgen05 接口。该接口增加了张量内存、基于描述符的布局、异步集体发射、mbarrier 事务记账和 proxy fence，其符号化效果与定序保证都需要建模。最后，我们讨论验证器可靠性证明中仍然存在的困难。

我们的贡献如下：

- 提出 TRITON AI COMPILER（TAIC）和 TRITON COMPILER ENVIRONMENT（TCENV）。它们把基于 LLM 的 PTX 生成，与编译反馈、正确性测试、基准测试、性能剖析和迭代优化结合起来，同时保持 kernel 的 ABI、启动配置、编译期常数和目标 GPU 契约。
- 表明生成的 PTX 在 12 个 kernel、三种架构上达到 Triton 的 0.83×–2.23×，使用了架构专用原语，并在近期 kernel 上超过 Triton，包括 B200 上的 BitDelta（3.34×）、FlashAttention（1.37×）和 Mamba-2（1.10×）。
- 这些结果共同暗示一种未来：AI 编译器可以用从高层源码到低层代码的端到端直接翻译，取代手写的中间表示，从而降低为新芯片和新架构建立软件所需的复杂度与时间。我们证明，要实现这一设想，需要强得多的验证，并通过把 Volta 扩展到 Blackwell，指出了仍然关键的困难。

2 相关工作

张量编译器、GPU DSL 与 kernel 库。 TVM 和 Ansor 把张量计算与 schedule 分开，并在 schedule 选择上搜索；Mirage 则在 kernel、线程块和线程三级做超优化。Triton 和 Halide 编译更高层的编程模型，CUTLASS 和 ThunderKittens 提供专家设计的实现与抽象。AI lowering 并不搜索人工设计的 schedule 空间，也不挑选某个库实现，而是从一个固定的 Triton kernel 直接合成针对硬件的 PTX。

LLM 生成的 GPU kernel。 KernelBench 和 TritonBench 评估的是从 PyTorch 级任务生成 CUDA 或 Triton。AI CUDA Engineer

加入了进化搜索，CUDA-L1 使用对比强化学习，Dr. Kernel 研究多轮强化学习，并设计了限制奖励投机和有偏梯度的方法。TritonGym 为智能体式 Triton 生成提供基准和工具编排框架，PTXBench 则生成带有规定架构专用 PTX 指令的 CUDA。我们的任务不同：对一个固定、自包含的 Triton 程序，只允许输出 PTX，并在编译契约之内不限制指令选择。

超优化与学习式编译。 STOKE 搜索无循环的 x86 二进制，SuperCoder 用 LLM 和强化学习优化更大的 x86-64 汇编程序。另一些工作建立在显式 IR 上：MLIR 提供多层编译器基础设施，CompilerGym 把编译器决策暴露给智能体，LLM Compiler 学习 LLVM IR 与汇编表示，CAKE 则共同设计面向智能体的 GPU IR 和编译器框架。与之相对，AI lowering 从 Triton 而不是目标汇编出发，绕过传统 IR 栈，并在 PTX 中同时完成指令选择、分配和调度。

生成代码的正确性。 KernelBench 及相关系统依赖有限的测试集。Volta 符号化执行 PTX 并证明等价，但不包括 Hopper 和 Blackwell 引入的 PTX 指令。

3 另一条编译路线

我们选择 Triton 到 PTX 来探索 AI lowering，是因为它提供了一条干净的、kernel 级的编译边界。若从 PyTorch 出发，会引入融合、分解和库分发等图级优化，使 kernel 级的收益更难分离。CUDA 则相反，许多低层选择已经显式暴露。Triton 处在有用的中间位置：它规定了计算和 kernel 接口，而把指令选择、线程映射、数据搬运、同步和调度等与硬件相关的决定留给编译器。

3.1 问题表述

一个 Triton kernel 在一份固定契约下编译，这份契约同时决定它的特化和执行。对每一次编译实例，特化对应于固定的值，例如张量维度、步长和 constexpr 参数（编译期取值）。运行时启动接口规定网格维度、线程数和参数 ABI。最后，生成的代码必须针对某一具体 GPU 架构（底层硬件）。AI lowering 必须保持这份契约，同时用模型生成的 PTX 替换传统的优化与 lowering 后端。

形式地，设 K 为一个 Triton kernel，(c, I, h) 表示其执行契约。其中 c 是固定的特化参数（编译期取值），I 是启动配置与参数 ABI（运行时启动接口），h 是目标 GPU 架构（底层硬件）。该契约在合法运行时输入 D 上诱导出参考的输入输出语义 $[\![K]\!]_{c,I,h}$；它并不指定唯一的一份 PTX 程序。候选 PTX 程序 P 正确，当且仅当它暴露所需的入口和参数 ABI，并且在同一契约 (c, I, h) 下满足

$$
\llbracket P \rrbracket_{c, I, h}(x) \approx_{\epsilon} \llbracket K \rrbracket_{c, I, h}(x), \quad \forall x \in \mathcal{D}. \tag{1}
$$

这里 $\approx_{epsilon}$ 表示在预先规定的绝对误差和相对误差之内数值等价；它允许浮点舍入的小差异，不要求按位相同。在形式化验证可用时，我们另外在把浮点运算抽象为实数算术之后检查精确相等。这一检查捕捉的是预期的算术语义，而不是编译器生成的某一特定浮点运算序列。

AI lowering 是迭代的合成过程，不是确定性的编译过程。在第 t 轮，框架用编译契约以及（若有）此前候选的反馈构造提示 $q_t = \mathrm{Prompt}(K, c, I, h, F_)$，再查询语言模型：

$$
P_t \sim p_{\theta}(\cdot \mid q_t), \tag{2}
$$

其中 p_{theta} 是模型的条件分布。

设 PTX 为 PTX 程序的空间。编译契约把 K 在固定的编译期取值 c、运行时接口 I 和目标 h 下的全部合法 lowering 定义为

$$
\mathcal{V}(K, c, I, h) = \left\{ P \in \mathrm{PTX} \;\middle|\; \begin{array}{l} \mathrm{Compiles}(P, h) \\ \wedge\; \mathrm{Safe}(P) \\ \wedge\; \mathrm{ABI}(P) = \mathrm{ABI}(K, I) \\ \wedge\; \forall x \in \mathcal{D}:\; \llbracket P \rrbracket_{c,I,h}(x) \approx_{\epsilon} \llbracket K \rrbracket_{c,I,h}(x) \end{array} \right\}, \tag{3}
$$

其中 $\mathrm{Compiles}(P,h)$ 要求 P 是目标 h 上的合法 PTX 程序，$\mathrm{Safe}(P)$ 要求内存安全且无竞争。设 $\hat{L}(P;h)$ 为固定基准协议估计的时延。

目标是 $\min_{P \in V(K,c,I,h)} \hat{L}(P;h)$。

下文把这一表述落实为可运行的系统。3.2 节介绍评估 PTX 的环境 TCENV。3.3 节描述执行迭代编译、并由 NCU 指导优化的框架 TAIC。4.5 节讨论验证，以及目前能够可靠验证的范围。

3.2 Triton 编译器环境：用于 AI lowering 的环境

我们构造 **TRITON COMPILER ENVIRONMENT（TCENV）**，一个受控环境，使 TAIC 可以只专注于 AI lowering。它把与 lowering 正交的配置细节抽象掉，例如编译期常数、网格维度、每块线程数和目标 GPU 架构：这些在编译时已经作为固定输入提供给 Triton，因此不必作为 AI lowering 的一部分去推断。具体而言，TCENV 暴露一个字典，其中包含 Triton kernel、其超参数和执行元数据，以及一份启动器无需再配置即可调用的 PTX 模板，函数签名和参数在 PTX 接口中保持不变。它还提供把生成的 PTX 汇编、启动、校验和分析所需的基础设施。因此 TAIC 收到的是高层 kernel 表示，产出的是语义等价的低层表示，如前一节所定义。图 2 把这条路径与标准编译流程并列画出。

人为偏弱的基线会把性能收益显得比实际大得多，这实际上是一种奖励投机。这一点重要，因为语义等价的实现在性能上可以差很多，Triton 教程中的融合 softmax 例子就是如此。因此，我们按照官方教程的优化实践来构造每个 Triton 参考实现；并且在 TCENV 暴露某个参考 kernel 之前，先在目标 GPU 上对它做自动调优，并在整个评估中使用所得配置。

TCENV 把随机化差分测试与可选的符号化验证（第 4.5 节）结合起来，以获得正确性信号。随机测试器生成会压到上溢、下溢、相消和精度损失的输入，并拒绝那些输出偏离参考、超出规定容差的 kernel。符号验证器则检查生成程序是否实现了预期计算。

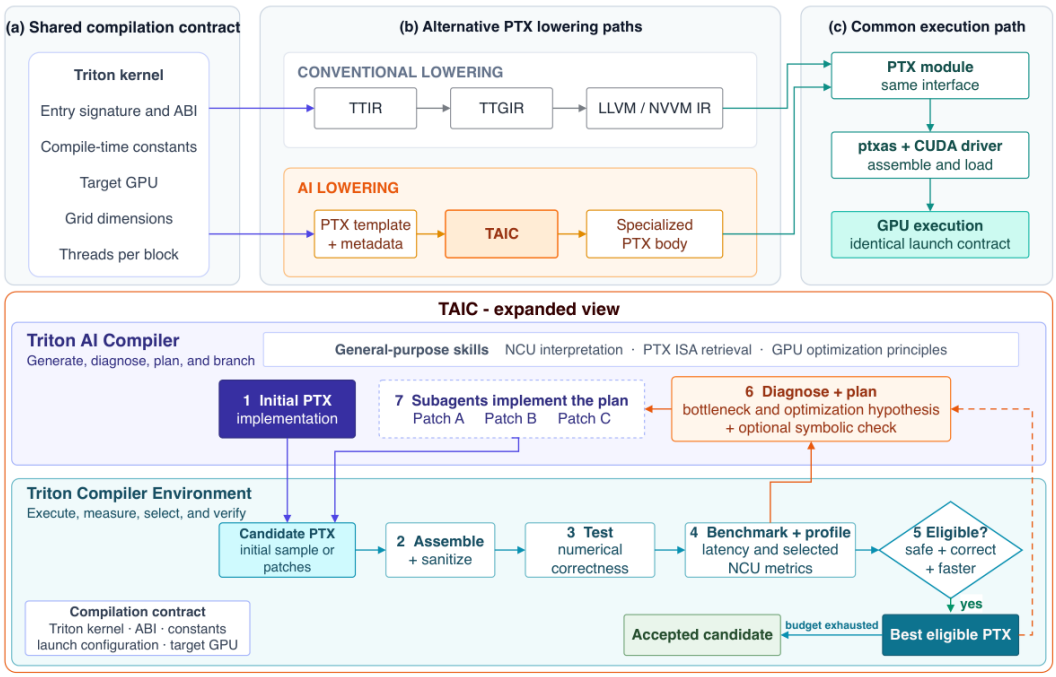


图 2. 系统总览与 AI 编译循环的细节。上：传统编译与 AI lowering 从同一份编译契约出发，产出接口兼容的 PTX 模块，并共享 NVIDIA 的执行路径。橙色块把 TAIC 展开为下方的详细循环：TAIC 提出并修改 PTX，TCENV 负责汇编、测试、基准、剖析和选择候选。

3.3 Triton AI 编译器：执行 AI lowering 的智能体

我们构造 **TRITON AI COMPILER（TAIC）**。它通过 TCENV（第 3.2 节）工作，实现一条直接的 Triton 到 PTX 编译路径，绕过 Triton 的中间编译器表示。TAIC 是一个 LLM，并配备三项通用技能：解读 Nsight Compute（NCU）指标、检索 PTX ISA 文档，以及应用高层 GPU 优化原则。它负责生成和优化，TCENV 提供执行与验证基础设施。这一职责划分在图 2 中写明。

TAIC 首先产出一份初始 PTX 实现，TCENV 对它汇编、做卫生检查、检查数值正确性并基准测试。受「验证器支持的推理时扩展」启发，我们接着细化它：给定当前 PTX，以及汇编与卫生检查结果、基准结果和选定的剖析观察，TAIC 诊断主要瓶颈并制定优化计划，范围从局部的指令级修改，到线程映射、数据搬运或同步的全局改变。子智能体独立地把该计划实现为候选补丁，TCENV 在同一契约下评估它们。只有能够汇编、通过安全与正确性门、并且改进了所测目标的候选，才有资格作为下一轮的种子。最好的合格实现及其测量反馈返回给 TAIC，它重复这一「诊断—计划—分叉—评估」循环，直到搜索预算耗尽。

4 评估

我们用 GPT-6 Astra 作为编译器 LLM。每个 kernel 在三代 NVIDIA GPU 上评估：L40S（Ada Lovelace，sm_89）、H100（Hopper，sm_90a）和 B200（Blackwell，sm_100a）。图 3 总结了这些目标上出现的架构专用特性。实验在除此之外空闲的 GPU 上顺序运行。生成的代码针对 PTX ISA 8.7、64 位地址空间，并按对应架构汇编。

	Ada	used?	Hopper	used?	Blackwell	used?
MMA	`mma.sync` + `ldmatrix`	✓	`wgmma` + TMA loads	✓	`tcgen05` + TMEM accum.	✓
SMEM	XOR bank swizzle	✓	128-B XOR swizzle	✓	64-B offset swizzle	✓
Copies	`cp.async` pipeline	✓	TMA + `mbarrier`	✓	TMA 2-D + phase parity	✓

图 3. 架构专用特性，以及它们在 LLM 生成的 PTX 中是否被用到。勾选表示生成的 kernel 在对应微架构上使用了该特性。Hopper 与 Blackwell 的条目（wgmma、TMA、mbarrier、tcgen05、TMEM）落在 Volta 原先已验证的语义之外；验证使用它们的 kernel，依赖于第 4.5 节的仿真器扩展。

	Ada	是否使用	Hopper	是否使用	Blackwell	是否使用
MMA	mma.sync + ldmatrix	是	wgmma + TMA 加载	是	tcgen05 + TMEM 累加	是
共享内存	XOR bank swizzle	是	128 字节 XOR swizzle	是	64 字节偏移 swizzle	是
拷贝	cp.async 流水	是	TMA + mbarrier	是	二维 TMA + 相位奇偶	是

基线使用 Triton 3.6.0，kernel 取自 TritonBench 和 OpenAI 官方 Triton 教程。为使基线有竞争力，我们在适用处进一步优化这些 kernel，包括在能获得更快实现时重写 kernel，以及在所评估的维度保证访问不越界时去掉边界掩码。对每个 kernel 和每块 GPU，我们报告通过自动调优找到的最快正确配置。为使比较代表标准的 Triton 编程方式，基线 kernel 不使用实验性 Triton 特性，也不手写内联汇编。

主评估全程使用 GPT-6 Astra，因为模型对比表明：有竞争力的 AI lowering 只在最新的前沿模型上出现。在 FP16 GEMM 上、相同协议和相同推理预算下，GPT-5.4 达到 Triton 的 0.73×，GPT-5.5 与 GPT-5.6 Terra 达到 0.87×，GPT-5.6 Luna 未能产出被接受的 kernel，GPT-5.6 Sol 达到 0.97×，而 GPT-6 Astra 是第一个与 Triton 持平（1.00×）的模型。由于 Triton 对这一工作负载已经提供了高度优化的基线，我们选择 GPT-6 Astra 作为实验的代表模型：在我们的对比中，它是第一个使直接 AI lowering 稳定地与传统编译竞争的模型。

我们评估的 kernel 覆盖四类对 LLM 性能关键的工作负载：注意力、GEMM、激活函数，以及归一化或正则化运算。它们从互补的方面压力测试 GPU 优化，包括 Tensor Core 利用率、归约与同步、内存访问和指令吞吐。问题规模固定，取 LLM 工作负载中常见的维度。

4.1 架构覆盖与对源码语义的理解

架构覆盖。 图 3 表明，生成的 PTX 使用的是每一代的现代机制，而不是最低公分母式的指令集：Ada 上是带 ldmatrix 的 mma.sync 和 cp.async 流水；Hopper 上是带 TMA 加载和 mbarrier 同步的 wgmma；Blackwell 上是带张量内存累加器和二维 TMA 传输的 tcgen05。共享内存布局同样从 XOR bank swizzle，进展到 128 字节 XOR 和 64 字节偏移 swizzle。这种覆盖也使验证变难：Hopper 和 Blackwell 的机制落在 Volta 原先验证过的语义之外，要到达它们，需要第 4.5 节的仿真器扩展。

从源码恢复语义。 因为我们的许多 Triton kernel 可能出现在模型的训练数据中，我们评估 TAIC 究竟是在恢复源码语义，还是在依赖记忆中的实现或熟悉的高层线索。我们首先测试随机生成的、大约 150 行的 kernel。它们不实现任何可识别的算法，在整数上运算以避免浮点差异，并行使广泛的 Triton 特性，包括整数与按位算术、取模和类似哈希的计算、tl.sum 与 tl.max 等归约、tl.static_range 循环、tl.where，以及 tl.multiple_of 和 tl.max_contiguous 等提示。TAIC 对每一个都产出了正确的 PTX。然后我们引入故意误导的线索：一个矩阵乘法，其累加被改成 $c[i,j] = a[i,k] \cdot b[k,j]$（原文如此，表示与常见乘加不同的写法）；一个 FlashAttention 变体，其 qk_scale = HEAD_DIM$^{-0.5}$ + 1.4426950408889634；以及一个名为 ReLU、但注释要求 GELU 的 kernel。这三种情况下，模型都遵循实际实现，而不是名字、注释或熟悉的算法结构，并产出语义等价的 PTX。最后，当 Triton 源码要求 IEEE-754 语义时，生成的 PTX 避免使用 Tensor Core 运算，进一步表明模型捕捉的是源码中表达的语义约束，而不是仅仅复现常见的优化模式。

4.2 常见 kernel 上的结果

图 4 表明，AI lowering 的收益强烈依赖于算子、精度和目标 GPU。Sigmoid、ReLU、SiLU 和 SwiGLU 落在 Triton 的大约 1% 以

内，RMSNorm 最多改进 4%。因此这些面板主要表明恢复了基线性能；它们很窄的纵轴不应被读成很大的收益。RoPE 最多改进 5%，矩阵—向量乘法在 0.98× 到 1.15× 之间。Softmax 改进 1.08–1.29×，卷积达到 1.17–2.23×。GEMM 更依赖于架构和精度：FP16 在 B200 和 L40S 上持平，但在 H100 上只有 0.83×；FP8 在 H100 上达到 1.61×，在 L40S 上 1.15×，在 B200 上大约持平。融合的 GEMM-SiLU kernel 呈现出同样的 H100 对比（FP16 为 0.96×，FP8 为 1.72×）。成本轴也区分了容易恢复的情形和需要大量搜索的情形：逐元素候选一般花费不到 1.50 美元，而卷积和 GEMM 候选大约需要 5 到 12 美元。主网格对每个实验报告单次运行；第 4.3 节另外把四个 B200 工作负载各重复五次。为给出 TAIC 所发现优化的具体例子，我们简要强调三个有代表性的 kernel。对归约求和，加速来自用架构专用的 Tensor Core 或 warp 归约算法替换 Triton 的通用归约。对卷积，加速来自在重叠窗口之间复用输入，并把打包的数据搬运与 Tensor Core 执行重叠。对 H100 FP8 GEMM，加速来自使用更大的 WGMMA tile、更少的同步点，以及更便宜的转置 epilogue。与 Triton 的详细比较见附录 A。

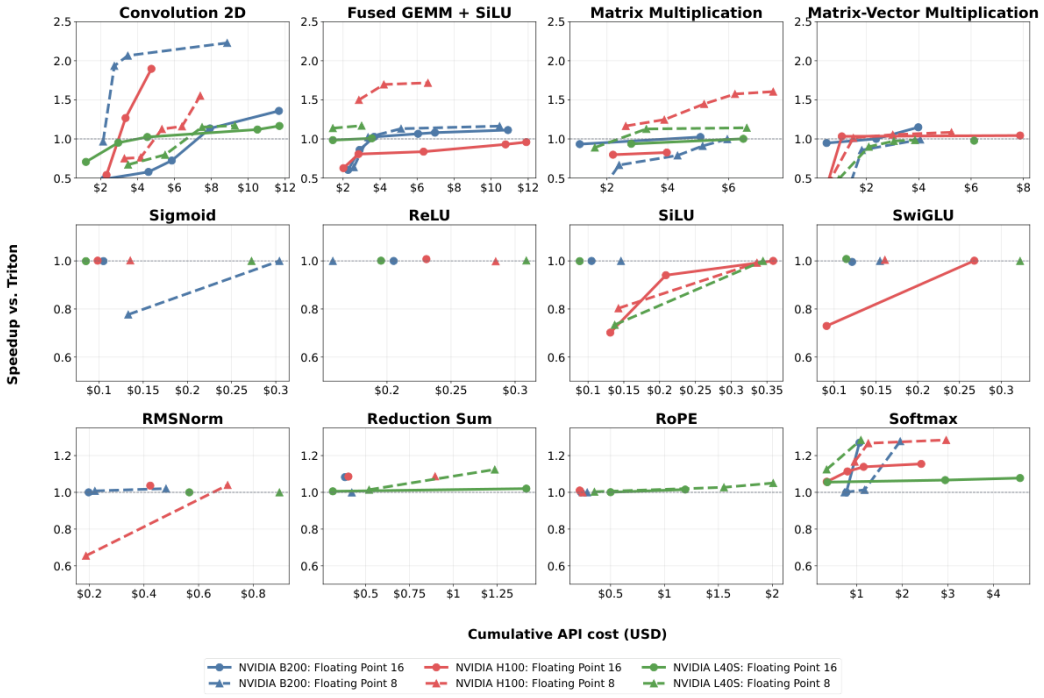


图 4. 相对 Triton，FP16 与 FP8 在三种 GPU 架构上的逐 kernel 加速 / 成本帕累托前沿。每个点是一个已验证的候选，它改进了在其累计 API 成本下所达到的最好加速；虚线表示与对应的、已自动调优的 Triton 基线持平。各面板的坐标轴独立缩放。

4.3 固定搜索预算下的可重复性

为测量运行之间的波动，我们把四个有代表性的 B200 实验各独立抽样重复五次。卷积、矩阵乘法和 FlashAttention 每次运行的 API 预算上限配置为 15 美元，较简单的逐元素 SwiGLU kernel 为 1 美元。预算门在发出每一次模型请求之前检查；因为一次请求是原子的，它完成后记录的最终成本可以超过所配置的门限。表 1 按时间顺序报告每次运行中最好的已验证候选。我们观察到 TRITON AI COMPILER 达到或超过 Triton 编译器，不过对那些相对基线有显著收益的 kernel，方差是中等的。

表 1. NVIDIA B200 上五次独立 TRITON AI COMPILER 运行的可重复性。数值是相对同一自动调优 Triton 基线的加速比。Budget 是每次运行配置的 API 成本门，在每次新的模型请求之前检查；进行中的请求可以在超过该门之后才结束。

Kernel	预算	第 1 次	第 2 次	第 3 次	第 4 次	第 5 次	均值 ± 标准差	最好
卷积（FP8）	15 美元	2.081×	1.706×	2.229×	1.933×	2.070×	2.004 ± 0.197	2.229×
矩阵乘法（FP16）	15 美元	1.002×	1.026×	1.032×	1.029×	1.024×	1.023 ± 0.012	1.032×
SwiGLU（FP16）	1 美元	1.000×	1.002×	1.000×	1.000×	1.000×	1.000 ± 0.001	1.002×
FlashAttention 前向（FP16）	15 美元	1.261×	1.075×	1.214×	1.263×	1.366×	1.236 ± 0.106	1.366×

4.4 近期发表的 kernel 上的结果

在第 4 节的 12 个 kernel 之外，我们评估近期机器学习论文中的 10 个 kernel：四种注意力变体（FlashAttention、FlashSinkhorn、Forgetting Attention、SageAttention），两种 BitDelta GEMM 变体（标准与批量矩阵乘法），来自 Lion 和 Dion

的两个内存受限 kernel，以及两个 Mamba-2 原语。Mamba-2 kernel 与注意力差别很大：它们传播紧凑的循环状态，而不是形成成对的 token 交互。我们使用作者官方的开源实现，不修改其算法或优化，只去掉不必要的掩码，并对相关参数做自动调优，以获得强的 Triton 基线。图 5 总结了结果。

在 15 美元预算下，AI lowering 在若干工作负载上大幅超过 Triton，最显著的是两种 BitDelta 变体（3.34× 和 2.51×）、FlashSinkhorn（1.55×）、SageAttention（1.41×）和 FlashAttention（1.37×）。内存受限和循环类工作负载仍接近持平，包括 Forgetting Attention（1.01×）、Lion（1.00×）、Dion（0.97×）和 Mamba-2（1.07× 与 1.10×）。因此，最大的收益出现在生成的 PTX 能够重组计算和数据搬运的时候。BitDelta 把打包的二值权重直接解码成 Tensor Core 操作数；FlashAttention 把完整的分数行映射到原生张量内存布局中的线程，从而避免跨 lane 归约；FlashSinkhorn 做独立的、warp 局部的 log-sum-exp 归约，并直接发射 FP16 MMA 指令。详细比较见附录 A。

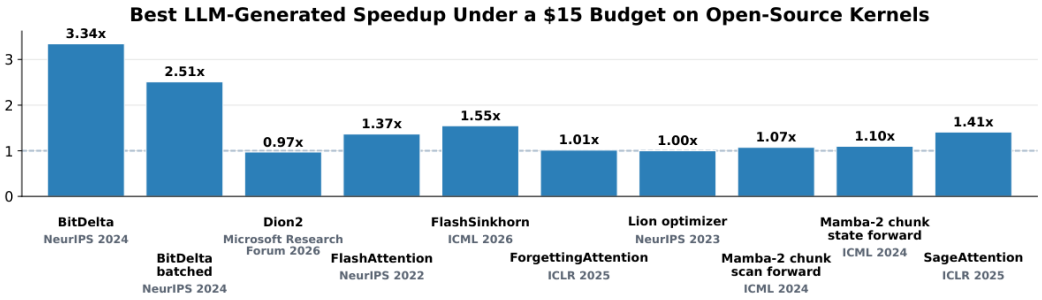


图 5. NVIDIA B200 上、数值验证通过的、最好的 LLM 生成 kernel。每根柱表示一个 kernel 的最好被接受候选；虚线表示与 Triton 持平。图题原文：15 美元预算下、开源 kernel 上 LLM 生成结果的最好加速比。

4.5 循环中的形式化验证

当我们在编译循环中启用 Volta 时，一个候选只有在通过数值检查、并且 Volta 证明它与 Triton 参考等价时才算有效。当 kernel 使用 Volta 支持范围之外的模式时，TCENV 把不支持的特性报告给 TAIC，由 TAIC 修改候选。表 2 表明，由此得到的 kernel 竞争力明显下降：在现代 GPU 上带来性能的那些特性，尚未全部被 Volta 支持。

为了能够使用现代 GPU 特性，我们扩展 Volta，以支持 TAIC 在不受约束地运行（循环中不使用 Volta）时所试图使用的、原先不支持的特性。下文和附录 B 描述这些特性：(i) 把数学常数作为立即数；(ii) 更多指令（例如 tanh 和 ldmatrix.trans）；(iii) 异步拷贝；(iv) 异步 Tensor Core 指令及其操作数搬运（tcgen05、wgmma、ldmatrix）；(v) 更广的屏障；(vi) 基于 shuffle 的归约；(vii) 打包格式，即多个较窄的值共享一个寄存器。这种支持不是免费的：Volta 的可信计算基从 4.2 万行增长到 6.0 万行，每一条新建模的指令都带来额外的不可靠性风险。细节见附录 B。

有了这些特性，我们可以验证 TAIC 不受限制的 PTX 输出（不在循环中用 Volta 约束它）：常见 kernel 套件中除归约求和与 softmax 之外的每一个，以及 10 个近期发表 kernel 中的 6 个（FlashAttention、Forgetting Attention、SageAttention、Lion，以及两个 Mamba-2 原语）。

三类构造仍然不支持：原子操作、依赖于输入的控制流，以及比特级的值操作。Volta 把浮点运算抽象为实数算术，因此不能推理那些通过直接操作比特编码来计算函数的代码。例如，TAIC 可以用整数 redux.sync 的 min/max 指令实现浮点 max 和 min 归约，方法是先对比特模式做感知符号的重映射，使整数序与浮点序一致。这一归约的正确性来自编码，而实数值抽象并不为编码建模。原子更新按设计让多个线程写同一位置；检查它们需要推理冲突更新的全部交错，这超出了 Volta 的执行模型。最后，Volta 针对的是没有输入相关控制流的、结构化 CTA kernel，因此排除了例如按输入范围选择不同公式的分段实现。

表 2. 在 NVIDIA B200、FP16 上，使用原始 Volta 时，最好的可证明验证实现与最好的数值验证实现的性能。

Kernel	可证明验证	数值验证
ReLU	1.00×	1.00×
GEMV	1.15×	1.15×
GEMM	0.24×	1.03×
FlashAttention	0.78×	1.37×
二维卷积	0.77×	1.35×

5 结论

我们表明，大语言模型可以把专门的 Triton kernel 直接 lower 成有竞争力的 PTX，并通过随机化差分测试。尽管生成可能比毫秒级的 Triton 编译贵几个数量级，这一成本只发生一次，并可以在 kernel 的重复执行上摊销。对支持尚少或全新的硬件，AI lowering 可能特别有吸引力：在那里，合成的代价可能远低于开发和维护一个专用的优化编译器后端。更广泛地说，在验证足够强的前提下，AI lowering 指向一种新范式，用来在硬件加速器上生成性能关键的代码。

参考文献

参考文献保持英文原文，便于检索。条目与 arXiv:2609.36800v1 第 10–12 页一致，此处不逐条改写。主要被引工作包括：TVM / Ansor、Mirage、Triton、Halide、CUTLASS、ThunderKittens、KernelBench、TritonBench、CUDA-L1、Dr. Kernel、Volta、MLIR、CompilerGym、LLM Compiler、CAKE、STOKE、SuperCoder、FlashAttention、BitDelta、Mamba-2、DeepGEMM。完整书目见原始 PDF。

附录 A 有代表性的优化例子

本节考察六个生成 PTX 超过 Triton 的代表性 kernel。前三个来自常见 kernel 套件，后三个来自在 B200 上评估的、近期发表的工作负载。

A.1 常见 kernel

归约求和：改变归约算法。 对 256×4096 的输入，Triton 把 tl.sum lower 成常规归约：在线程和 warp 内累加，通过共享内存合并部分结果，最后使用原子更新。生成的 PTX 则把每一行的和表达为矩阵形的计算，并使用打包的 FP16 或 FP8 输入、FP32 累加的 Tensor Core。在 H100 FP16 上，两条 $16 \times 8 \times 16$ 的 mma.sync 指令替换了标量转换和归约链；随后三级 shuffle-and-add 归约 MMA 的输出，每个 warp 一次原子更新，没有共享内存归约，也没有块级屏障。在 L40S FP8 上，单个 warp 使用一条 $16 \times 8 \times 32$ 的 MMA，达到 Triton 的 1.125×。在 H100 FP8 上，kernel 改为缩放 E4M3 输入，用 redux.sync.add 计算整数值的部分和，并用一次屏障合并四个 warp 的和，达到 1.09×。

卷积：复用重叠的输入窗口。 卷积把 $32 \times 64 \times 56 \times 56$ 的输入和 $128 \times 64 \times 3 \times 3$ 的滤波器映射到 $32 \times 128 \times 54 \times 54$ 的输出。Triton 把它 lower 成 Tensor Core 上的隐式 GEMM，归约维为 576，但使用标量加载和共享内存暂存。生成的 kernel 则复用相邻 3×3 窗口所共享的值。在 H100 FP16 上，32 位加载取相邻像素，漏斗移位构造相邻的值，打包的共享内存存储和双缓冲把操作数准备与 WGMMA 重叠，得到 1.90× 加速。在 B200 FP8 上，字节置换在若干滤波器偏移之间复用一小集 32 位加载，而流水化的输入与权重缓冲供给 tcgen05.mma，得到 2.23×。在 L40S FP8 上，环形共享内存缓存提供类似的复用，达到 1.17×。这些 kernel 保留与 Triton 相同的 Tensor Core 计算，但通过输入复用、打包访问和更深的流水，消除了冗余的 gather。

H100 FP8 GEMM：降低 WGMMA 与 epilogue 开销。 生成的 kernel 使用更大的 wgmma.mma.async.m64n256k32，而 Triton 使用 m64n64k32，因此要覆盖同一输出 tile 需要更多 WGMMA 指令。显式双缓冲在当前 Tensor Core 运算执行时准备下一块 A/B tile，同步只发生在某一级共享内存被消费或复用之时。epilogue 也不同：Triton 反复把累加器片段存到共享内存、同步、再重新加载以获得所需布局；生成的 kernel 使用转置的 stmatrix 变体，随后一次同步和向量化的全局存储。这些选择减少了指令、同步和布局转换的开销，解释了 Hopper 上的收益，以及其他架构上没有这一收益的原因。

A.2 近期发表的 kernel

这些例子把每个生成实现与同一次运行中的 Triton PTX 比较。它们表明，收益来自在目标层同时重新考虑指令序列、数据搬运、同步和分解，而不是仅仅孤立地直接发出 PTX。

BitDelta：把二值权重直接解码成 Tensor Core 操作数。 对所评估的 64×128 输出 tile，Triton 拆开每个二值权重，用整数算术计算 $2b\text{-}1$，把结果转换成 FP16，并把解码后的权重暂存在共享内存。生成的 PTX 直接构造 $\mathrm{pm}\ 1$ 的打包 FP16 表示：一次比特置换把打包权重与 FP16 的符号位置对齐，此后每个 lop3 通过修改一个打包常数的符号位，产生两个解码值。它还转置内部的矩阵乘，使解码后的权重可以直接写入 tcgen05.mma 的张量内存操作数，从而去掉一次共享内存的存—重载循环。四级激活流水、两个交替的权重缓冲，以及从 32 提到 64 的归约步长，进一步减少了暂存、同步和循环开销。所得 kernel 达到 Triton 的 3.34×，批量变体用同一策略达到 2.51×。

FlashAttention：让 softmax 的归属与张量内存布局对齐。 两种实现都用 Blackwell Tensor Core 做 $\mathrm{QK}^{\mathrm{mathsf\,T}}$ 和 PV，但中间的

softmax 组织方式不同。Triton 把每一行分数分布到两个 lane 上，因此需要跨 lane 归约。生成的 PTX 在原生张量内存布局中让每个线程拥有完整的 32 个分数的一行，使最大值与求和成为寄存器局部的归约，并消除 shuffle 指令。它还把两个 64 行的 query tile 合成一个 128 行的 tile，于是每一块 key/value tile 服务两倍数量的 query。打包的 f32x2 算术减少了标量工作，而双缓冲以及归一化更新与矩阵运算之间的重叠，把每次循环迭代的块级屏障从三个减到一个。这些改变合在一起得到 1.37× 加速。

FlashSinkhorn：把归约划分到相互独立的 warp。 对 $n = m = 1024$、坐标维 64，Triton 做 32 次顺序的 log-sum-exp 更新，八个 warp 在每个 32×32 tile 上协作。生成的 PTX 给每个 warp 分配一个独立的 128 点切片，并保持分开的最大值与求和累加器。每个 warp 执行八次迭代，循环中没有块级屏障或 shuffle，部分结果只在最后合并一次。固定的坐标 tile 留在寄存器中，原始 FP16 坐标直接送入 mma.sync.m16n8k16，替换了 Triton 的 FP16 到 FP32 转换和 TF32 的 m16n8k8 运算。每个 MMA 的归约长度加倍，也使每次点积的矩阵指令数减半。所得 kernel 通过评估容差，并达到 Triton 的 1.55×。

附录 B 验证与 Volta 扩展

Volta 所支持的特性集合，不足以验证为近期 GPU 架构优化的 PTX 程序的正确性（见表 2）。作为概念验证，我们扩展 Volta，以支持 AI 模型最常发出的指令。我们并不声称这些扩展是可靠的或完备的。下面描述这些扩展，其形式化留作未来工作。

主要增加的是对异步操作的支持。异步拷贝（cp.async）一类指令引入了一类新的内存危险，超出了同步内存访问上的跨线程数据竞争。Volta 以字节粒度跟踪数据竞争，方法是跟踪同步屏障之间读者线程和写者线程的访问。我们处理异步数据搬运的方式，是同时在源和目的上跟踪异步操作。一次异步操作，从其发起指令起，直到一条适当的屏障指令执行，都被视为在飞行中。若一次异步数据搬运与一次同步操作或另一次异步操作重叠，扩展后的 Volta 就报告内存危险。

下一项重要增加，是支持生成 kernel 中常见的那条长尾指令。其中 Tensor Core 运算，例如 tcgen05、wgmma 和 ldmatrix 指令族，因为配置集合很大、又与异步操作交互，所以最为复杂。我们进一步实现了缺失的数学算子，例如 tanh 和 exp2，并把它们用指数表达，以便利用已有的求解器化简规则。我们增加对打包表示的支持，例如 16x2 和 8x4 打包值，以及其上的算术、转换和加载 / 存储操作。

最后，我们发现有必要显式支持常见模式。例如在 shuffle 归约中，所有线程可能同时把同一个值写到同一个内存地址。这种定义良好的数据竞争在 kernel 中很常见，并且在语义上无害，因为每个线程存储的是同一个值。为此，我们修改 Volta，在它检查竞争时先确立被写入的值彼此相等，也就是在符号化执行的中途调用等价检查器。

我们的扩展足以生成达到或超过 Triton 编译器的 kernel，但仍有重大的内在局限。首先，Volta 不支持依赖于数据的控制流。尽管少见，数据相关的控制流仍会出现在 kernel 中，例如分段多项式近似，或流式计算中的条件更新。原子操作也不在范围内，以避免必须考虑所有可能的线程交错。最后，由于使用无限精度的浮点表示，那些依赖浮点数比特布局性质的 kernel，即使在有限精度算术下会产生预期结果，仍可能被 Volta 拒绝。